

By Feng-Li Lian, James R. Moyne, and Dawn M. Tilbury

# Performance Evaluation of Control Networks:

## Ethernet, ControlNet, and DeviceNet

**T**he traditional communication architecture for control systems, which has been successfully implemented in industry for decades, is point to point. Expanding physical setups and functionality, however, are pushing the limits of the point-to-point architecture. Hence, a traditional centralized point-to-point control system is no longer suitable to meet new requirements such as modularity, decentralization of control, integrated diagnostics, quick and easy maintenance, and low cost [1]. Network systems with common bus architectures, called networked control systems (NCSs), provide several advantages such as small volume of wiring and distributed processing. Especially with respect to manufacturing systems, NCS architectures that utilize processing power at each node allow for the modularization of functionality and standard interfaces for interchangeability and interoperability. Object-based device models separate generic, device-type-specific from manufacturer-specific functionality.

Many different network types have been promoted for use in control systems. In this article, we compare three of them: the Ethernet bus, with carrier sense multiple access



©2000 Image 100 Ltd.

with collision detection (CSMA/CD), token-passing bus (e.g., ControlNet), and controller area network (CAN) bus (e.g., DeviceNet). We consider how each control network could be used as a communication backbone for a networked control system connecting sensors, actuators, and controllers. A detailed discussion of the medium access control (MAC) sublayer protocol for each network is provided. For each protocol, we study the key parameters of the corresponding network when used in a control situation, including network utilization, magnitude of the expected time delay, and characteristics of time delays. Simulation results are presented

*Lian and Tilbury (tilbury@umich.edu) are with the Department of Mechanical Engineering, University of Michigan, 2250 G.G. Brown Building, 2350 Hayward St., Ann Arbor, MI 48109-2125, U.S.A. Moyne is with the Department of Electrical Engineering and Computer Science, University of Michigan, Ann Arbor, MI 48109-2125, U.S.A.*

for several different scenarios, and the advantages and disadvantages of each network are summarized.

## Performance Objectives

Distributed systems contain a large number of interconnected devices to perform the desired operations over a large area of coverage; examples include industrial automation, building automation, office and home automation, intelligent vehicle systems, and advanced aircraft and spacecraft [2]-[6]. *Data networks* or *control networks* may be used, depending on the information exchanged. *Data networks* are characterized by large data packets, relatively infrequent bursty transmission, and high data rates; they generally do not have hard real-time constraints. *Control networks*, in contrast, must shuttle countless small but frequent packets among a relatively large set of nodes to meet the time-critical requirements. The key element that distinguishes control networks from data networks is the capability to support real-time or time-critical applications [7].

With NCSs, decision and control functions (including signal processing) can be distributed among controllers on the network. When designing an NCS, however, a new constraint must be accommodated—the limited bandwidth of the communication network. The effective bandwidth of a network is defined as the maximum amount of meaningful data that can be transmitted per unit time, exclusive of headers, padding, stuffing, etc.; this contrasts with the more traditional definition of network bandwidth, which is the number of raw bits transmitted per unit time. Four factors affect the availability and utilization of the network bandwidth: the sampling rates at which the various devices send information over the network, the number of elements that require synchronous operation, the data or message size of the information, and the MAC sublayer protocol that controls the information transmission [8]. Therefore, to achieve the timing constraints and guarantee the performance of NCSs, network traffic optimization algorithms and controller design for the devices connected to the network must be analyzed.

The performance metrics of network systems that impact control system requirements include access delay, transmission time, response time, message delay, message collisions (percentage of collisions), message throughput (percentage of packets discarded), packet size, network utilization, and determinism boundaries. For control systems, candidate control networks generally must meet two main criteria: bounded time delay and guaranteed transmission; that is, a message should be transmitted successfully within a bounded time delay. Unsuccessfully transmitted or large time-delay messages from a sensor to an actuator, for example, may deteriorate system performance or make systems unstable. Several protocols have been proposed to meet these requirements for control systems. They include Ethernet (IEEE 802.3:CSMA/CD), token bus (IEEE 802.4), token ring (IEEE 802.5), and CAN (CSMA/AMP). Control networks are typically based on one of two medium access protocols:

| Nomenclature.<br>All variables defined in this article are time varying. |                                                |
|--------------------------------------------------------------------------|------------------------------------------------|
| $N_{\text{data}}$                                                        | Data size (bytes).                             |
| $N_{\text{ovhd}}$                                                        | Overhead size (bytes).                         |
| $N_{\text{pad}}$                                                         | Padding frame (bytes).                         |
| $N_{\text{stuff}}$                                                       | Stuffing frame (bytes).                        |
| $P_{\text{eff}}$                                                         | Network efficiency (%).                        |
| $P_{\text{util}}$                                                        | Network utilization (%).                       |
| $T_{\text{block}}$                                                       | Blocking time at the source node.              |
| $T_{\text{bit}}$                                                         | Bit time.                                      |
| $T_{\text{dcode}}$                                                       | Decoding time at the destination node.         |
| $T_{\text{dcomp}}$                                                       | Computing time at the destination node.        |
| $T_{\text{delay}}$                                                       | Time delay of a message.                       |
| $T_{\text{frame}}$                                                       | Frame time on the network medium.              |
| $T_{\text{guard}}$                                                       | Guardband period.                              |
| $T_{\text{max}}$                                                         | Maximum simulation time.                       |
| $T_{\text{node}}$                                                        | Maximum token holding time.                    |
| $T_{\text{peri}}$                                                        | Message period.                                |
| $T_{\text{retx}}$                                                        | Retransmission time on the network medium.     |
| $T_{\text{post}}$                                                        | Postprocessing time at the destination node.   |
| $T_{\text{pre}}$                                                         | Preprocessing time at the source node.         |
| $T_{\text{prop}}$                                                        | Propagation time on the network medium.        |
| $T_{\text{queue}}$                                                       | Queuing time at the source node.               |
| $T_{\text{resid}}$                                                       | Residual time.                                 |
| $T_{\text{scode}}$                                                       | Encoding time at the source node.              |
| $T_{\text{scomp}}$                                                       | Computing time at the source node.             |
| $T_{\text{tx}}$                                                          | Transmission time on the network medium.       |
| $T_{\text{token}}$                                                       | Token-passing time from one node to the other. |
| $T_{\text{wait}}$                                                        | Waiting time at the source node.               |
| $\mathcal{N}_{\text{hp}}$                                                | The set of high-priority nodes in DeviceNet.   |
| $\mathcal{N}_{\text{node}}$                                              | The set of nodes on the network.               |
| $\mathcal{N}_{\text{noqueue}}$                                           | The set of nodes without messages queued.      |
| $\mathcal{N}_{\text{queue}}$                                             | The set of nodes with messages queued.         |
| $\mathbf{T}_{\text{dest}}$                                               | Time instant at destination node.              |
| $\mathbf{T}_{\text{src}}$                                                | Time instant at source node.                   |

CAN, used by smart distributed system (SDS), DeviceNet [9], and CAN Kingdom; and the token ring or bus, used by process field bus (PROFIBUS) [10], manufacturing automation protocol (MAP) [11], ControlNet [12], and fiber distributed data interface (FDDI) [13].

## Control Network Basics

In this section, we discuss the MAC sublayer protocol of three candidate control networks: Ethernet, ControlNet (a token-bus-based network), and DeviceNet (a CAN-based network). (Note that Ethernet is not a complete protocol so-

lution but only a MAC sublayer definition, whereas ControlNet and DeviceNet are complete protocol solutions. Following popular usage, we use the term "Ethernet" to refer to Ethernet-based complete network solutions.) The MAC sublayer protocol, which describes the protocol for obtaining access to the network, is responsible for satisfying the time-critical/real-time response requirement over the network and for the quality and reliability of the communication between network nodes [14]. Our discussion and comparison thus focuses on the MAC sublayer protocols.

transmit, it listens to the network. If the network is busy, it waits until the network is idle; otherwise it transmits immediately. If two or more nodes listen to the idle network and decide to transmit simultaneously, the messages of these transmitting nodes collide and the messages are corrupted. While transmitting, a node must also listen to detect a message collision. On detecting a collision between two or more messages, a transmitting node stops transmitting and waits a random length of time to retry its transmission. This random time is determined by the standard binary exponential backoff (BEB) algorithm: the retransmission time is randomly chosen between 0 and  $(2^i - 1)$  slot times, where  $i$  denotes the  $i$ th collision event detected by the node and one slot time is the minimum time needed for a round-trip transmission. After ten collisions have been reached, however, the interval is fixed at a maximum of 1023 slots. After 16 collisions, the node stops attempting to transmit and reports failure back to the node microprocessor. Further recovery may be attempted in higher layers [17].

## Ethernet has almost no delay at low network loads.

The Ethernet frame format is shown in Fig. 1 [17], [18]. The total overhead is 26 bytes. The data packet frame size is between 46 and 1500 bytes. There is a nonzero minimum data size requirement because the standard states that valid frames must be at least 64 bytes long from destination address to checksum (72 bytes, including preamble and start of delimiter). If the data portion of a frame is less than 46 bytes, the pad field is used to fill out the frame to the minimum size. There are two reasons for this minimum size limitation. First, it makes it easier to distinguish valid frames from "garbage." When a transceiver detects a collision, it truncates the current frame, which means that stray bits and pieces of frames frequently appear on the cable. Second, it prevents a node from completing the transmission of a short frame before the first bit has reached the far end of cable, where it may collide with another frame. For a 10-Mb/s Ethernet with a maximum length of 2500 m and four repeaters, the minimum allowed frame time or slot time is 51.2  $\mu$ s, which is the time required to transmit 64 bytes at 10 Mb/s [17].

### Ethernet (CSMA/CD)

Ethernet uses the CSMA/CD mechanism for resolving contention on the communication medium. The CSMA/CD protocol is specified in the IEEE 802.3 network standard and is described briefly as follows [15]-[17]. When a node wants to

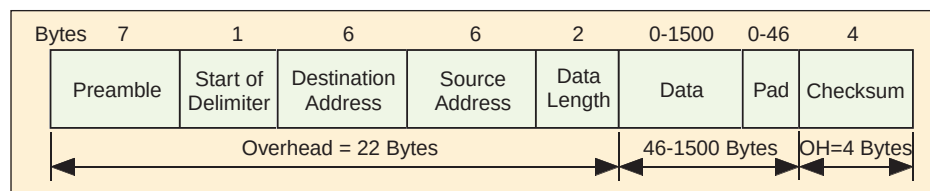


Figure 1. Ethernet (CSMA/CD) frame format.

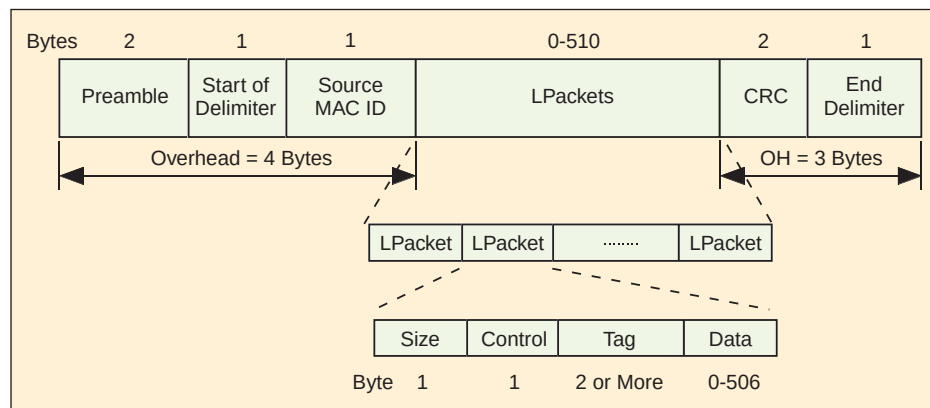


Figure 2. The message frame of ControlNet.

Advantages

Because of low medium access overhead, Ethernet uses a simple algorithm for operation of the network and has almost no delay at low network loads [11]. No communication bandwidth is used to gain access to the network compared with the token bus or token ring protocol. Ethernet used as a control net-

work commonly uses the 10 Mb/s standard (e.g., Modbus/TCP); a high-speed (100 Mb/s or even 1 Gb/s) Ethernet is mainly used in data networks [17], [22].

## Disadvantages

Ethernet is a nondeterministic protocol and does not support any message prioritization. At high network loads, message collisions are a major problem because they greatly affect data throughput and time delay, which may be unbounded [11], [20]. The Ethernet capture effect existing in the standard BEB algorithm, in which a node transmits packets exclusively for a prolonged time despite other nodes waiting for medium access, causes unfairness and substantial performance degradation [21]. Based on the BEB algorithm, a message may be discarded after a series of collisions; therefore, end-to-end communication is not guaranteed. Because of the required minimum valid frame size, Ethernet uses a large message size to transmit a small amount of data.

Several solutions have been proposed for using Ethernet in control applications. For example, every message could be time stamped before it is sent. This requires clock synchronization, however, which is not easy to accomplish, especially with a network of this type [22]. Various schemes based on deterministic retransmission delays for the collided packets of a CSMA/CD protocol result in an upper-bounded delay for all the transmitted packets. However, this is achieved at the expense of inferior performance to CSMA/CD at low to moderate channel utilization in terms of delay throughput [14]. Other solutions also try to prioritize CSMA/CD (e.g., LonWorks) to improve the response time of critical packets [23]. Using switched Ethernet by subdividing the network architecture is another way to increase its efficiency [17].

## ControlNet (Token-Passing Bus)

MAP, PROFIBUS, and ControlNet are typical examples of token-passing bus control networks. These are deterministic networks because the maximum waiting time before sending a message frame can be characterized by the token rotation time. The token bus protocol (IEEE 802.4) allows a linear, multidrop, tree-shaped, or segmented topology [11]. The nodes in the token bus network are arranged logically into a ring and, in the case of ControlNet, each node knows the address of its predecessor and its successor. During operation of the network, the node with the token transmits data frames until either it runs out of data frames to transmit or the time it has held the token reaches a limit. The node then regenerates the token and transmits it to its logical successor on the network. If a node has no message to send, it just passes the token to the successor node. The physical location of the successor is not important because the token is sent to the logical neighbor.

Data frames do not collide, as only one node can transmit at a time. The protocol also guarantees a maximum time between network accesses for each node and has provisions to regenerate the token if the token holder stops transmitting and does not pass the token to its successor. Nodes can also be added dynamically to the bus and can request to be dropped from the logical ring.

The message frame format of ControlNet is shown in Fig. 2 [12]. The total overhead is 7 bytes, including preamble, start delimiter, source MAC ID, cyclic redundancy check (or CRC), and end delimiter. The data packet frame, also called the link packet (Lpacket) frame, may include several Lpackets that contain size, control, tag, and data fields with total frame size between 0 and 510 bytes. The individual destination address is specified within the tag field. The size field specifies the number of byte pairs (from three to 255) contained in an individual Lpacket, including the size, control, tag, and link data fields.

The ControlNet protocol adopts an implicit token-passing mechanism and assigns a unique MAC ID (from one to 99) to each node. As in general token-passing buses, the node with the token can send data; however, there is no real token passing around the network. Instead, each node monitors the source MAC ID of each message frame received. At the end of a message frame, each node sets an "implicit token register" to the received source MAC ID + 1. If the implicit token register is equal to the node's own MAC ID, that node may now transmit messages. All nodes have the same value in their implicit token registers, preventing collisions on the medium. If a node has no data to send, it just sends a message with an empty Lpacket field, called a null frame.

The length of a cycle, called the network update time (NUT) in ControlNet or token rotation time (TRT) in general, is divided into three major parts: scheduled, unscheduled, and guardband, as shown in Fig. 3. During the scheduled part of an NUT, each node can transmit time-critical/scheduled data by obtaining the implicit token from zero to  $S$ . During the unscheduled part of an NUT, nodes zero to  $U$  share the opportunity to transmit non-time-critical data in a

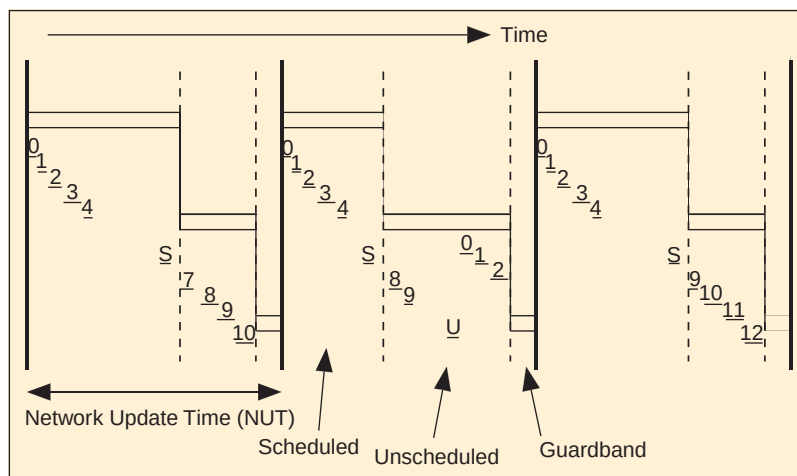


Figure 3. Medium access during scheduled, unscheduled, and guardband time.

round-robin fashion until the allocated unscheduled duration is expired. When the guardband time is reached, all nodes stop transmitting, and only the node with the lowest MAC ID, called the “moderator,” can transmit a maintenance message, called the “moderator frame,” which accomplishes the synchronization of all timers inside each node and the publishing of critical link parameters such as NUT, node time,  $S$ ,  $U$ , etc. If the moderator frame is not heard for two consecutive NUTs, the node with the lowest MAC ID will begin transmitting the moderator frame in the guardband of the third NUT. Moreover, if a moderator node notices that another node has a lower MAC ID than its own, it immediately cancels its moderator role.

### Advantages

The token bus protocol is a deterministic protocol that provides excellent throughput and efficiency at high network loads [11], [14]. During network operation, the token bus can dynamically add nodes to or remove nodes from the network. This contrasts with the token ring case, where the nodes physically form a ring and cannot be added or removed dynamically [11]. Scheduled and unscheduled segments in each NUT cycle make ControlNet suitable for both time-critical and non-time-critical messages.

### Disadvantages

Although the token bus protocol is efficient and deterministic at high network loads, at low channel traffic its performance cannot match that of contention protocols. In general, when there are many nodes in one logical ring, a large percentage of the network time is used in passing the token between nodes when data traffic is light [14].

### DeviceNet (CAN Bus)

CAN is a serial communication protocol developed mainly for applications in the automotive industry but is also capable of offering good performance in other time-critical industrial applications. The CAN protocol is optimized for short messages and uses a CSMA/arbitration on message priority (CSMA/AMP) medium access method. Thus the protocol is message oriented, and each

message has a specific priority that is used to arbitrate access to the bus in case of simultaneous transmission. The bit stream of a transmission is synchronized on the start bit, and the arbitration is performed on the following message identifier, in which a logic zero is dominant over a logic one. A node that wants to transmit a message waits until the bus is free and then starts to send the identifier of its message bit by bit. Conflicts for access to the bus are resolved during transmission by an arbitration process at the bit level of the arbitration field, which is the initial part of each frame. Hence, if two devices want to send messages at the same time, they first continue to send the message frames and then listen to the network. If one of them receives a bit different from the one it sends out, it loses the right to continue to send its message, and the other wins the arbitration. With this method, an ongoing transmission is never corrupted.

In a CAN-based network, data are transmitted and received using message frames that carry data from a transmitting node to one or more receiving nodes. Transmitted data do not necessarily contain addresses of either the source or the destination of the message. Instead, each message is labeled by an identifier that is unique throughout the network. All other nodes on the network receive the message and accept or reject it, depending on the configuration of mask filters for the identifier. This mode of operation is known as multicast.

DeviceNet is a relatively low-cost communication link connecting devices to a network; it has received considerable acceptance in device-level manufacturing applications. The DeviceNet specification is based on standard CAN (11-bit identifier only) with an additional application and physical layer specification [9], [19]. The CAN protocol supports two message frame formats: standard CAN (version 2.0A, 11-bit identifier) and extended CAN (version 2.0B, 29-bit identifier).

The frame format of DeviceNet is shown in Fig. 4 [9]. The total overhead is 47 bits, which includes start of frame (SOF), arbitration (11-bit identifier), control, CRC, acknowledgment (ACK), end of frame (EOF), and intermission (INT) fields. The size of a data field is between 0 and 8 bytes. The DeviceNet

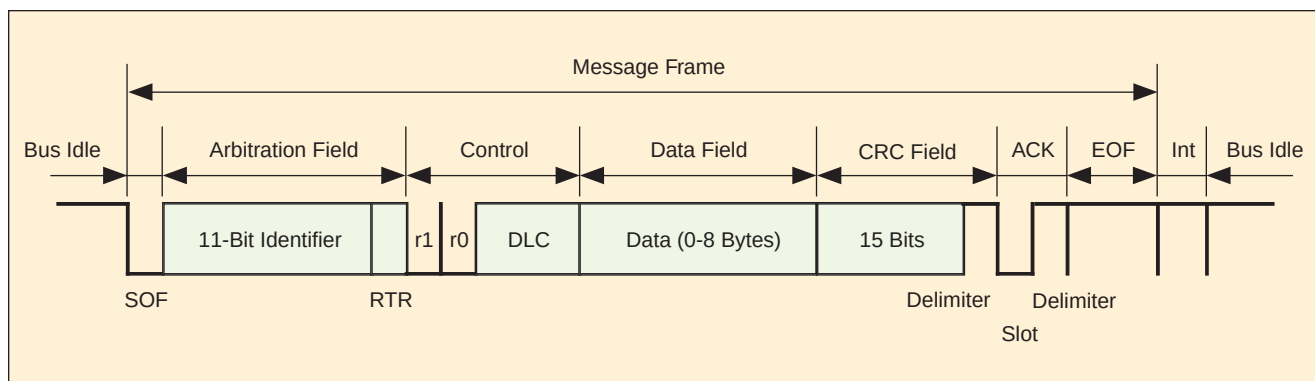
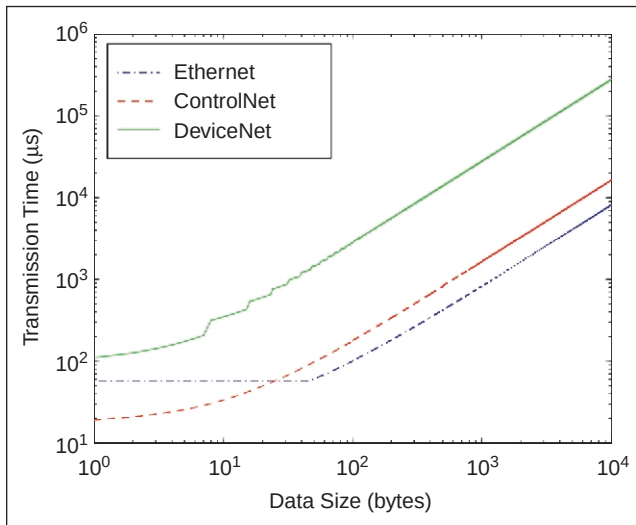


Figure 4. The message frame format of DeviceNet.





**Figure 5.** A comparison of the transmission time versus the data size for the three networks.

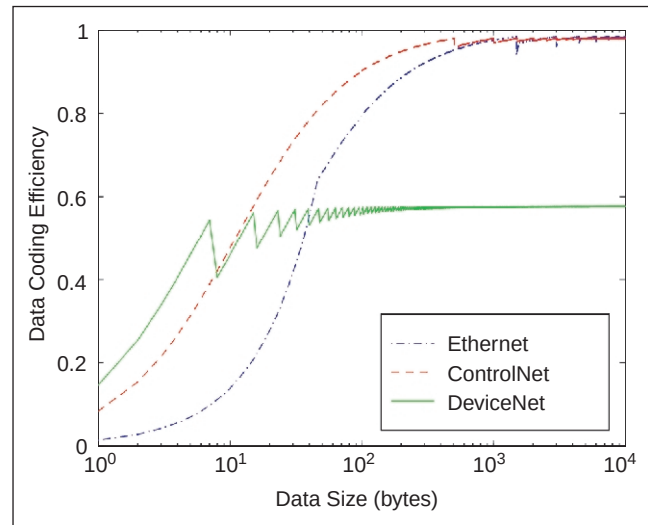
protocol uses the arbitration field to provide source and destination addressing as well as message prioritization.

### Advantages

CAN is a deterministic protocol optimized for short messages. The message priority is specified in the arbitration field. Higher priority messages always gain access to the medium during arbitration. Therefore, the transmission delay for higher priority messages can be guaranteed.

### Disadvantages

The major disadvantage of CAN compared with the other networks is the slow data rate (maximum of 500 Kb/s). Thus the throughput is limited compared with other control networks. The bit-synchronization requirement of the CAN protocol also limits the maximum length of a DeviceNet network. CAN is also not suitable for transmission of mes-



**Figure 6.** A comparison of the data coding efficiency versus the data size for each network.

sages of large data sizes, although it does support fragmentation of data that is more than 8 bytes.

## Comparative Analysis of Three Control Networks

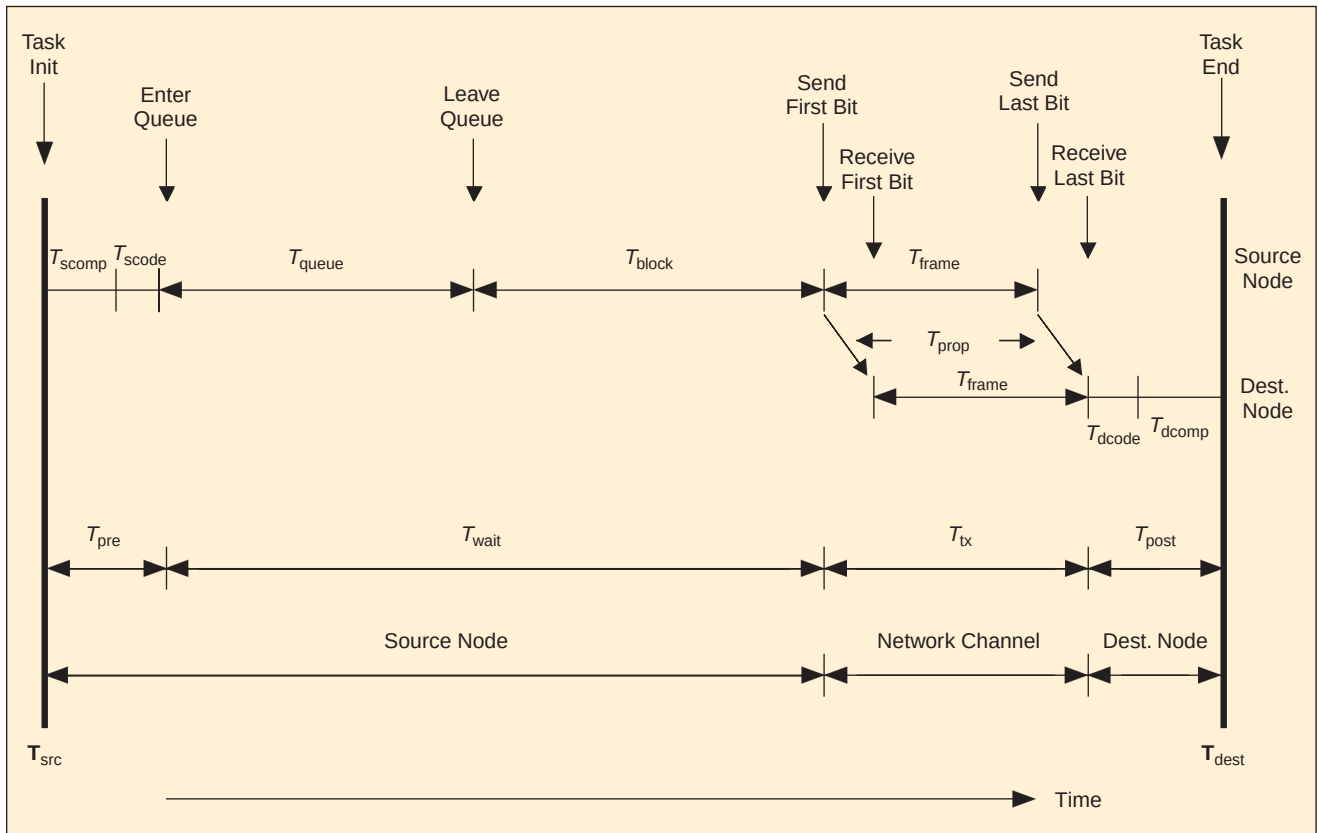
Two aspects of the three control networks discussed here are compared in Figs. 5 and 6. The parameters used in these figures are listed in Table 1.

As shown in Fig. 5, the transmission time for DeviceNet is longer than the others because of the lower data rate (500 Kb/s). Ethernet requires less transmission time on larger data sizes (> 20 bytes) compared with the others. Although ControlNet uses less time to transmit the same amount of data, it needs some time (NUT) to gain access to the network.

Fig. 6 shows the data coding efficiency of the three control networks versus the data size. The data coding effi-

**Table 1.** Typical system parameters of control networks.

|                                                                                                                                              | Ethernet                         | ControlNet | DeviceNet         |
|----------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------|------------|-------------------|
| Data rate <sup>a</sup> (Mb/s)                                                                                                                | 10                               | 5          | 0.5               |
| Bit time (μs)                                                                                                                                | 0.1                              | 0.2        | 2                 |
| Max. length (m)                                                                                                                              | 2500                             | 1000       | 100               |
| Max. data size (bytes)                                                                                                                       | 1500                             | 504        | 8                 |
| Min. message size <sup>b</sup> (byte)                                                                                                        | 72 <sup>c</sup>                  | 7          | 47/8 <sup>d</sup> |
| Max. number of nodes                                                                                                                         | >1000                            | 99         | 64                |
| Typical Tx speed (m/s)                                                                                                                       | Coaxial cable: 2×10 <sup>8</sup> |            |                   |
| a: typical data rate;<br>b: zero data size;<br>c: including the preamble and start of delimiter fields;<br>d: DeviceNet overhead is 47 bits. |                                  |            |                   |



**Figure 7.** A timing diagram showing time spent sending a message from a source node to a destination node.  $T_{\text{delay}} = T_{\text{dest}} - T_{\text{src}} = T_{\text{pre}} + T_{\text{wait}} + T_{\text{tx}} + T_{\text{post}}$ . Delays occur at the source node due to computation and coding of the message, queuing at the source node, and blocking due to network traffic. Once the message is sent, there is a propagation time delay (due to the physical length of the network) and a transmission time delay (due to message size). At the destination node, there are again decoding and computation delays before the data can be used.

ciency is defined by the ratio of the data size and the message size (i.e., the total number of bytes used to transmit the data). For small data sizes, DeviceNet is the best among these three types and Ethernet is the worst. For large data sizes, ControlNet and Ethernet are better than DeviceNet (DeviceNet is only 58% efficient, but ControlNet and Ethernet are nearly 98% efficient for large data size transmission). For control systems, the data size is generally small. Therefore, the above analysis suggests that DeviceNet may be preferable in spite of the slow data rate. Before making that decision, however, the average and total time delay and the throughput of the network must be investigated.

The discontinuities seen in Figs. 5 and 6 are caused by data fragmentation (i.e., the maximum size limitation per message). The maximum data sizes are 1500, 504, and 8 bytes for Ethernet, ControlNet, and DeviceNet, respectively. The flat portion of the Ethernet plot for small data sizes in Fig. 5 is due to the minimum data size requirement (46 bytes).

## Timing Analysis of Control Networks

In this section, we characterize the time delays of the three control networks by studying their timing parameters. Fig. 7

shows a general timing diagram of the initialization and ending of the task of sending a message over the network. The time delay of a message,  $T_{\text{delay}}$ , is defined as the difference between the time when the source node begins the process of sending a message,  $T_{\text{src}}$ , and the time when the destination node completes reception of this message,  $T_{\text{dest}}$  (i.e.,  $T_{\text{delay}} = T_{\text{dest}} - T_{\text{src}}$ ).

The total time delay can be broken into three parts: time delays at the source node, on the network channel, and at the destination node, as shown in Fig. 7. The time delay at the source node includes the preprocessing time,  $T_{\text{pre}}$ , which is the sum of the computation time,  $T_{\text{scomp}}$ , the encoding time,  $T_{\text{scode}}$ , and the waiting time,  $T_{\text{wait}}$ , which is the sum of the queue time,  $T_{\text{queue}}$ , and the blocking time,  $T_{\text{block}}$ . Depending on the amount of data the source node must send and the traffic on the network, the waiting time may be significant. The network time delay includes the total transmission time of a message and the propagation delay of the network. This will depend on message size, data rate, and the length of the network cable. The time delay at the destination node is the postprocessing time,  $T_{\text{post}}$ , which is the sum of the decoding time,  $T_{\text{dcode}}$ , and the computation time,  $T_{\text{dcomp}}$ , at the destination node. The time delay can be explicitly expressed by

$$\begin{aligned}
T_{\text{delay}} &= \mathbf{T}_{\text{dest}} - \mathbf{T}_{\text{src}} \\
&= T_{\text{pre}} + T_{\text{wait}} + T_{\text{tx}} + T_{\text{post}} \\
&= \underbrace{T_{\text{scomp}} + T_{\text{scode}}}_{T_{\text{pre}}} + \underbrace{T_{\text{queue}} + T_{\text{block}}}_{T_{\text{wait}}} + \underbrace{T_{\text{frame}} + T_{\text{prop}}}_{T_{\text{tx}}} \\
&\quad + \underbrace{T_{\text{dcode}} + T_{\text{dcomp}}}_{T_{\text{post}}}.
\end{aligned} \tag{1}$$

Because the preprocessing and postprocessing times are typically constant compared with the waiting time and the transmission time and are a limitation of computer processing parameters rather than network physical and protocol parameters, we will ignore them in the following discussion. The queuing time,  $T_{\text{queue}}$ , is the time a message waits in the buffer at the source node while previous messages in the queue are sent. It depends on the blocking time of previous messages in queue, the periodicity of messages, and the processing load. Although  $T_{\text{queue}}$  is difficult to analyze, we include it in our simulations. In some control applications, old messages are discarded, effectively setting  $T_{\text{queue}}$  to zero; however, this strategy may require a non-standard network protocol. In the following subsections, we will analyze the blocking time, frame time, and propagation time for each of the three candidate control networks.

## Blocking Time

The blocking time, which is the time a message must wait once a node is ready to send it, depends on the network protocol and is a major factor in the determinism and performance of a control network. It includes waiting time while other nodes are sending messages and the time needed to resend the message if a collision occurs.

## Ethernet Blocking Time

We first consider the blocking time for Ethernet, which includes the time taken by collisions with other messages and the subsequent time waiting to be retransmitted. The BEB algorithm described earlier indicates a probabilistic waiting time. An exact analysis of expected blocking time delay for Ethernet is very difficult [24]. At a high level, the expected blocking time can be described by

$$E\{T_{\text{block}}\} = \sum_{k=1}^{16} E\{T_k\} + T_{\text{resid}} \tag{2}$$

where  $T_{\text{resid}}$  denotes the residual time until the network is idle, and  $E\{T_k\}$  is the expected time of the  $k$ th collision.  $E\{T_k\}$  depends on the number of backlogged and unbacklogged nodes as well as the message arrival rate at each node. For the sixteenth collision, the node discards this message and reports an error message to the higher

level processing units [17]. It can be seen that  $T_{\text{block}}$  is not deterministic and may be unbounded due to the discarding of messages.

## ControlNet Blocking Time

In ControlNet, if a node wants to send a message, it must wait to receive the token from the logically previous node. Therefore, the blocking time,  $T_{\text{block}}$ , can be expressed by the transmission times and TRTs of previous nodes. The general formula for  $T_{\text{block}}$  can be described by

**The token bus protocol provides excellent throughput and efficiency at high network loads.**

$$T_{\text{block}} = T_{\text{resid}} + \sum_{j \in N_{\text{noqueue}}} T_{\text{token}}^{(j)} + \sum_{j \in N_{\text{queue}}} \min(T_{\text{tx}}^{(j, n_j)}, T_{\text{node}}) + T_{\text{guard}} \tag{3}$$

where  $T_{\text{resid}}$  is the residual time needed by the current node to finish transmitting,  $N_{\text{noqueue}}$  and  $N_{\text{queue}}$  denote the sets of nodes with messages and without messages in the queues, respectively, and  $T_{\text{guard}}$  is the time spent on the guardband period, as defined earlier. For example, if node 10 is waiting for the token, node 4 is holding the token and sending messages, and nodes 6, 7, and 8 have messages in their queues, then  $N_{\text{noqueue}} = \{5, 9\}$  and  $N_{\text{queue}} = \{4, 6, 7, 8\}$ . Let  $n_j$  denote the number of messages queued in the  $j$ th node, and let  $T_{\text{node}}$  be the maximum possible time (i.e., token holding time) assigned to each node to fully utilize the network channel; for example, in ControlNet  $T_{\text{node}} = 8272 \mu\text{s}$ , which is a function of the maximum data size, overhead frame size, and other network parameters.  $T_{\text{token}}$  is the token-passing time, which depends on the time needed to transmit a token and the propagation time from node  $i-1$  to node  $i$ . ControlNet uses an implicit token, and  $T_{\text{token}}$  is simply the sum of  $T_{\text{frame}}$  with zero data size and  $T_{\text{prop}}$ . If a new message is queued for sending at a node while that node is holding the token, then  $T_{\text{block}} = T_{\text{tx}}^{(j, n_j)}$ , where  $j$  is the node number. In the worst case, if there are  $N$  master nodes on the bus and each one has multiple messages to send, which means each node uses the maximum token holding time, then  $T_{\text{block}} = \sum_{i \in N_{\text{node}} \setminus \{j\}} \min(T_{\text{tx}}^{(i, n_i)}, T_{\text{node}})$ , where the min function is used because, even if it has more messages to send, a node cannot hold the token longer than  $T_{\text{node}}$  (i.e.,  $T_{\text{tx}}^{(j, n_j)} \leq T_{\text{node}}$ ). ControlNet is a deterministic network because the maximum time delay is bounded and can be characterized by (3). If the periods of each node and message are known, we can explicitly describe the sets  $N_{\text{noqueue}}$  and  $N_{\text{queue}}$  and  $n_j$ . Hence,  $T_{\text{block}}$  in (3) can be determined explicitly.



## DeviceNet Blocking Time

If the blocking time,  $T_{\text{block}}$ , in DeviceNet is finite, it can be found by iterating the following equation from  $k=1$  until it converges [25]:

$$T_{\text{block}}^{(k)} = T_{\text{resid}} + \sum_{j \in N_{\text{hp}}} \left\lceil \frac{T_{\text{block}}^{(k-1)} + T_{\text{bit}}}{T_{\text{peri}}^{(j)}} \right\rceil T_{\text{tx}}^{(j)} \quad (4)$$

where  $T_{\text{resid}}$  is the residual time needed by the current node to finish transmitting,  $N_{\text{hp}}$  is the set of nodes with higher priority than the waiting node,  $T_{\text{peri}}^{(j)}$  is the period of the  $j$ th node, and  $\lceil x \rceil$  denotes the smallest integer that is greater than or equal to  $x$ . The summation denotes the time needed to send all the higher priority messages. For a low-priority node, while it is waiting for the channel to become available, it is possible for other high-priority nodes to be queued, in which case the low-priority node loses the arbitration again. This situation accumulates the total blocking time. The worst-case  $T_{\text{resid}}$  under a low traffic load is

$$T_{\text{resid}} = \max_{j \in N_{\text{node}}} T_{\text{tx}}^{(j)} \quad (5)$$

where  $N_{\text{node}}$  is the set of nodes on the network. Because of the priority-arbitration mechanism, however, low-priority message transmission may not be deterministic or bounded under high loading.

## Frame Time

The frame time,  $T_{\text{frame}}$ , depends on the size of the data, the overhead, any padding, and the bit time. Let  $N_{\text{data}}$  be the size of the data in terms of bytes,  $N_{\text{ovhd}}$  be the number of bytes used as overhead,  $N_{\text{pad}}$  be the number of bytes used to pad the remaining part of the frame to meet the minimum frame size requirement, and  $N_{\text{stuff}}$  be the number of bytes used in a stuffing mechanism (on some protocols). The frame time can then be expressed by

$$T_{\text{frame}} = \left[ N_{\text{data}} + N_{\text{ovhd}} + N_{\text{pad}} + N_{\text{stuff}} \right] \times 8 \times T_{\text{bit}}. \quad (6)$$

The values  $N_{\text{data}}$ ,  $N_{\text{ovhd}}$ ,  $N_{\text{pad}}$ , and  $N_{\text{stuff}}$  can be explicitly described for Ethernet, ControlNet, and DeviceNet protocols [24]. (The bit-stuffing mechanism in DeviceNet is as follows: If more than 5 bits in a row are "1," then a "0" is added, and vice versa. Ethernet and ControlNet use Manchester biphasic encoding and therefore do not require bit-stuffing.)

## Propagation Time

The propagation time,  $T_{\text{prop}}$ , depends on the signal transmission speed and the distance between the source and destination nodes. For the worst case, the propagation delays from one end to the other of the network cable for these three control networks are  $T_{\text{prop}} = 25.6 \mu\text{s}$  for Ethernet (2500 m),  $T_{\text{prop}} = 10 \mu\text{s}$  for ControlNet (1000 m), and  $T_{\text{prop}} = 1 \mu\text{s}$  for

DeviceNet (100 m). The length in parentheses represents the typical maximum cable length used. The propagation delay is not easily characterized because the distance between the source and destination nodes is not constant among different transmissions. For comparison, we will assume that the propagation times of these three network types are the same, say,  $T_{\text{prop}} = 1 \mu\text{s}$  (100 m). Note that  $T_{\text{prop}}$  in DeviceNet is generally less than one bit time because DeviceNet is a bit-synchronized network. Hence, the maximum cable length is used to guarantee the bit synchronization among nodes.

## Case Studies

In this section, we define critical network parameters and then study two cases of NCSs: a control network system with ten nodes, each with 8 bytes of data to send every period, and an SAE vehicle example with 53 nodes [25]. MATLAB is used to simulate the MAC sublayer protocols of the three control networks. Network parameters such as the number of nodes, the message periods, and message sizes can be specified in the simulation model. In our study, these network parameters are constant. The simulation program records the time delay history of each message and calculates network performance statistics such as the average time delay seen by messages on the network, the efficiency and utilization of the network, and the number of messages that remain unsent at the end of the simulation run.

For control network operation, the message connection type must be specified. There are three types of message connections: strobe, poll, and change of state (COS)/cyclic. In a *strobe* connection, the master device broadcasts a strobed message to a group of devices and these devices respond with their current condition. In this case, all devices are considered to sample new information at the same time. In a *poll* connection, the master sends individual messages to the polled devices and requests update information from them. Devices only respond with new signals after they have received a poll message. *COS/cyclic* devices send out messages either when their status is changed (COS) or periodically (cyclic). Although COS/cyclic seems most appropriate from the traditional control systems point of view, strobe and poll are commonly used in industrial control networks [9].

Based on these different types of message connections, we consider the following three releasing policies. The first policy, which we call the "zero releasing policy," assumes every node tries to send its first message at  $t = 0$  and sends a new message every period. This type of situation occurs when a system powers up and there has been no pre-scheduling of messages or when there is a strobe request from the master. The second policy, namely, the "random releasing policy," assumes a random start time for each node; each node still sends a new message every period. A possible situation for this releasing policy is the COS or cy-

click messaging where no preschedule is done. In the third policy, called “scheduled releasing policy,” the start-send time is scheduled to occur (to the extent possible) when the network is available to the node; this occurs in a polled connection.

In addition to varying the release policy, we also change the period of each node to demonstrate the effect of traffic load on the network. For each releasing policy and period, we calculate the average time delays of these ten nodes and the efficiency and utilization of the three different control networks; we also record the number of unsent and failure or discarded messages for each network. Further, we examine the effect of node numbering on the network performance. We then compare the simulation results to the analytic results described above. For ControlNet and DeviceNet, the maximum time delay can be explicitly determined. For Ethernet, the expected value of the time delay can be computed using the BEB algorithm once the releasing policy is known.

### Total and Average Time Delays

For a given running time, say  $T_{\max} = 10$  s, we can calculate the total and average time delays from each node for each network

$$T_{\text{delay}}^{\text{sum}} = \sum_{i \in N_{\text{node}}} \sum_{j=1}^{M^{(i)}} T_{\text{delay}}^{(i,j)} \quad (7)$$

$$T_{\text{delay}}^{\text{avg}} = \frac{1}{N} \sum_{i \in N_{\text{node}}} \left[ \frac{\sum_{j=1}^{M^{(i)}} T_{\text{delay}}^{(i,j)}}{M^{(i)}} \right] \quad (8)$$

where  $N$  is the total number of nodes,  $N_{\text{node}}$  is the set of nodes, and  $M^{(i)}$  is the number of messages requested at node  $i$ . We assume all messages are periodic; thus the total

## CAN is a deterministic protocol optimized for short messages.

number of messages is equal to the total running time divided by the period of messages (i.e.,  $M^{(i)} = \lfloor T_{\max} / T_{\text{peri}}^{(i)} \rfloor$ , where  $\lfloor x \rfloor$  denotes the smallest integer less than equal to  $x$ ). The average delay can be computed for the entire network, as shown, or for the  $i$ th node.

### Network Efficiency

We will define the efficiency of a network,  $P_{\text{eff}}$ , as the ratio of the total transmitting time to the time used to send messages, including queuing time, blocking time, and so on; that is,

$$P_{\text{eff}} = \frac{\sum_{i \in N_{\text{node}}} \sum_{j=1}^{M^{(i)}} T_{\text{tx}}^{(i,j)}}{T_{\text{delay}}^{\text{sum}}} \quad (9)$$

Therefore,  $P_{\text{eff}} \rightarrow 1$  denotes that all the time delay is due to the transmission delay and the network performance is

Table 2. Simulation result of three releasing policies with message period of 5000  $\mu\text{s}$  (ten-node case).

| Releasing Policies                                   | Zero | Random | Scheduled |
|------------------------------------------------------|------|--------|-----------|
| <b>Average time delay (<math>\mu\text{s}</math>)</b> |      |        |           |
| Ethernet                                             | 1081 | 172    | 58        |
| ControlNet                                           | 241  | 151    | 32        |
| DeviceNet                                            | 1221 | 620    | 222       |
| <b>Efficiency (%)</b>                                |      |        |           |
| Ethernet                                             | 5.3  | 33.0   | 100       |
| ControlNet                                           | 13.3 | 21.1   | 100       |
| DeviceNet                                            | 18.2 | 35.8   | 100       |
| <b>Utilization (%)</b>                               |      |        |           |
| Ethernet                                             | 34.5 | 16.4   | 11.5      |
| ControlNet                                           | 6.4  | 6.4    | 6.4       |
| DeviceNet                                            | 44.4 | 44.4   | 44.4      |

good. On the other hand,  $P_{\text{eff}} \rightarrow 0$  means that most of the time delay is due to message contention or collision.

## Network Utilization

The utilization of a network,  $P_{\text{util}}$ , is defined by the ratio of the total time used to transmit data and the total running time; that is,

$$P_{\text{util}} = \frac{\sum_{i \in N_{\text{node}}} \sum_{j=1}^{M^{(i)}} (T_{\text{tx}}^{(i,j)} + T_{\text{retx}}^{(i,j)})}{T_{\text{max}}} \quad (10)$$

where  $T_{\text{retx}}^{(i,j)}$  is the time taken to retransmit the  $(i,j)$ th message.  $P_{\text{util}}$  describes the percentage of effective bandwidth used by the nodes or, conversely, the utilization of the network. If  $P_{\text{util}} \rightarrow 0$ , there is sufficient bandwidth left on the network for other purposes. If  $P_{\text{util}} \rightarrow 1$ , the network is saturated, and we have to redesign the network layout or reassign the traffic load. Note that for Ethernet, under high loading conditions,  $P_{\text{util}}$  can approach one. However, effective data communication can approach zero (i.e.,  $P_{\text{eff}} \rightarrow 0$ ) because  $P_{\text{util}}$  is dominated by  $T_{\text{retx}}^{(i,j)}$ .

## Number of Unsent Messages

Control systems need required information to be transmitted successfully and immediately. If the information transmission on a network induces lost messages, the system perfor-

mance may deteriorate or even become unstable. Hence, it is very important to evaluate a network protocol according to the number or the possibility of unsent messages.

## Stability of NCSs

The stability of a network itself is defined by the number of messages in the queue of each node. If this number is larger than a certain constant or tends to infinity as time increases, the network is said to be unstable (even though we assume infinite buffer size). On the other hand, the stability of a networked control system should be defined by the performance of both the network and the control system. That is, when the networks are unstable (i.e., increasing queue lengths) and the network-induced time delays degrade the control system sufficiently, the networked control system can become unstable. Note, however, that it is possible to have a system with an unstable network but a stable control system, and vice versa. In this study, we only consider the stability of a network.

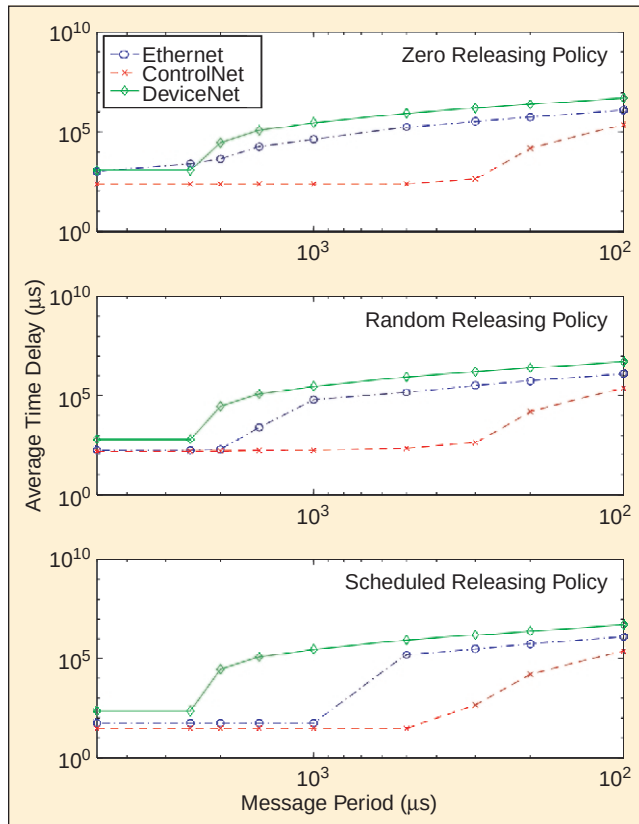
If sensors sample the data faster than the network can transmit the data, then the network will be saturated and data will be queued at the buffer (unless it is discarded). In designing an NCS, both the effective bandwidth and the sensors' sampling rate must be considered. Although high sampling rates improve performance in traditional control systems, they also induce high traffic loads on the network medium. High traffic loads increase the message time delays and can degrade control performance. A preliminary case study on the tradeoff between sampling time and network traffic has been performed [26].

## Case Study I: Ten Nodes, Equal Periods

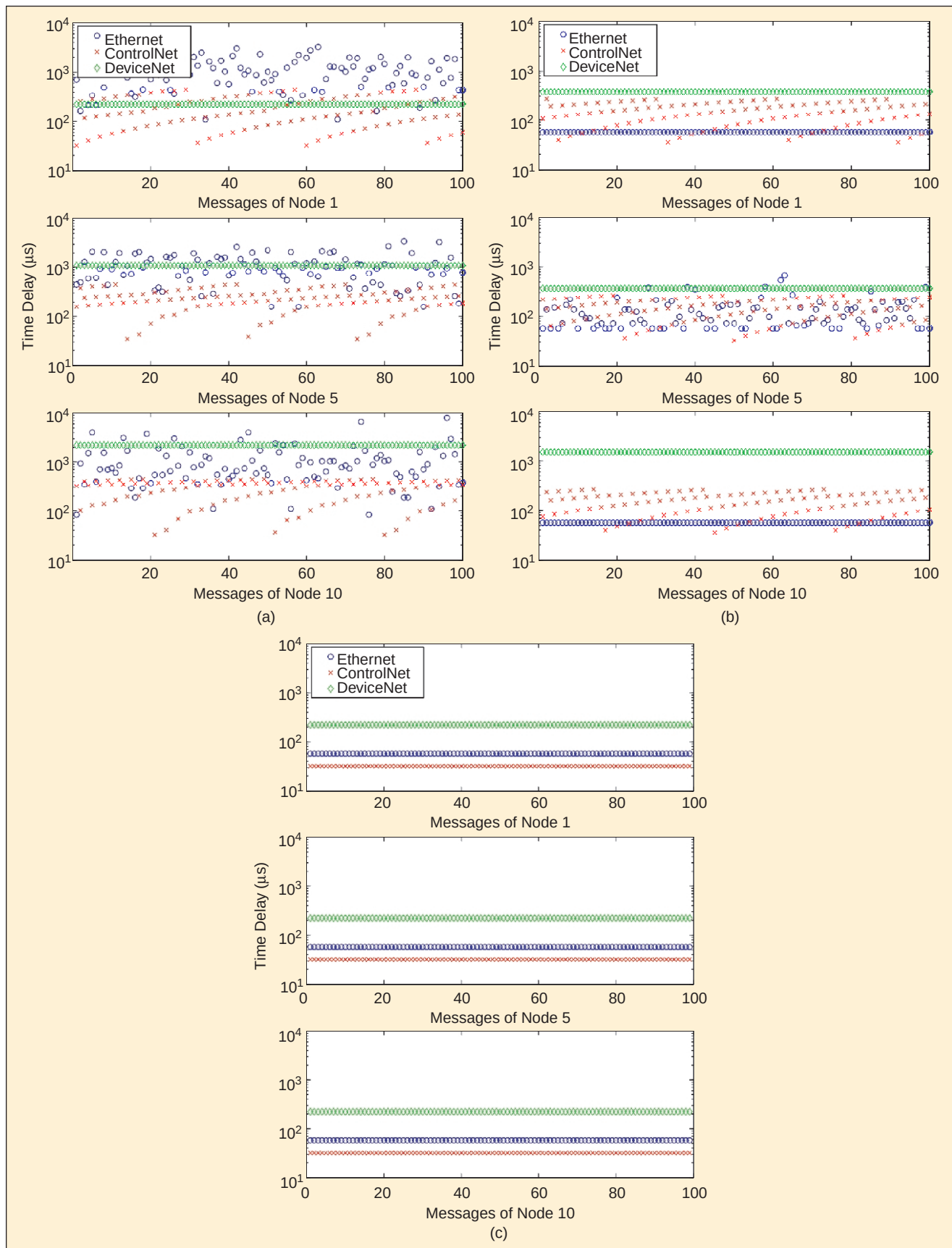
In this case study of a network with ten nodes with equal periods and data sizes, we examine nine different periods with a total simulation time of 0.5 s. The periods were chosen based on the total time to send all ten messages over the different networks; 576, 320, and 2200  $\mu\text{s}$  for Ethernet, ControlNet, and DeviceNet, respectively. The three releasing policies (zero, random, scheduled) were also examined. The simulation results for a message period of 5000  $\mu\text{s}$  are summarized in Table 2 and Figs. 8-10.

When the message period is longer than the total message transmission time, the performance indexes are similar for all three networks. Because of the different network data rates, however, the message period at which saturation occurs is different for the three networks. When the period is shorter than the total transmission time (i.e., the traffic load exceeds the availability of the network), the utilization approaches 100% and the time delay increases (it will become unbounded as the simulation time goes to infinity). Therefore, the network becomes unstable.

The average time delays for the three releasing policies are shown in Fig. 8. The zero releasing policy has the longest average delay in every network because all nodes experience contention when trying to send messages. Although

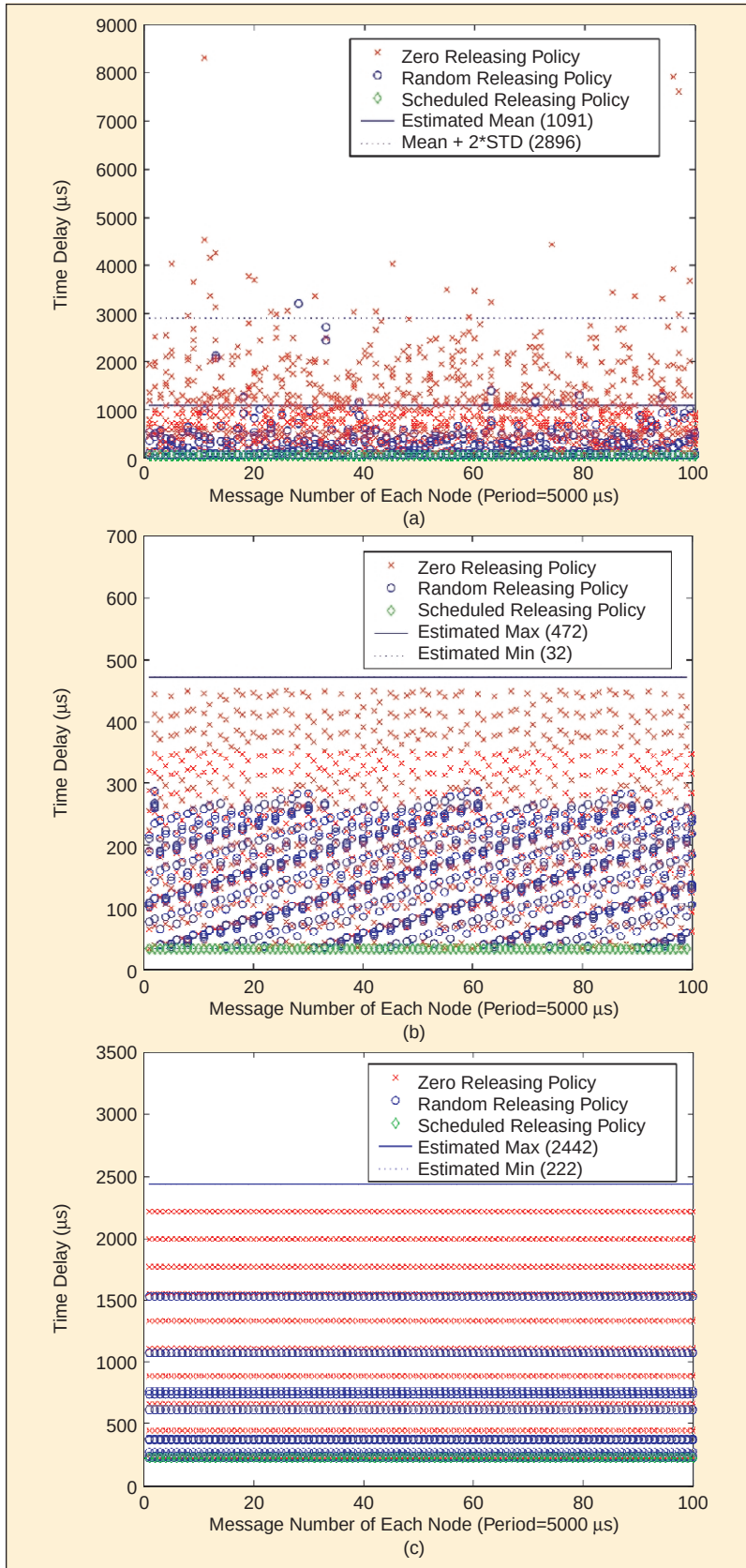


**Figure 8.** A comparison of the average time delay versus message period (ten-node case).



**Figure 9.** The time delay history of nodes one, five, and ten with a period of 5000 s and using the three releasing policies (ten-node case); (a) zero releasing policy, (b) random releasing policy, and (c) scheduled releasing policy.





**Figure 10.** Message time delay associated with three releasing policies (ten-node case). The estimated mean, maximum, and minimum values are computed from the network analysis for the zero and scheduled releasing policies; (a) Ethernet, (b) ControlNet, and (c) DeviceNet.

the Ethernet data rate is much faster than DeviceNet's, the delays due to collisions and the large required message size combine to increase the average time delay for Ethernet in this case. For a typical random releasing policy, average time delays are reduced because not all nodes try to send messages (or experience network contention) at the same time, although some contention still exists. The scheduled releasing policy makes the best use of each individual network; the average time delay of this releasing policy is only the transmission time. When the networks are not saturated, the average time delay is just equal to the frame time (i.e., the time taken to transmit a message frame). In this case, all three networks maintain a constant time delay.

For the two deterministic control networks (i.e., DeviceNet and ControlNet), if the network is not saturated (i.e.,  $P_{util} < 100\%$ ), there are no messages queued at the buffer. For Ethernet, however, there are always some messages queued at the buffer, and, moreover, some messages are discarded due to the BEB algorithm. We also notice that the average time delay of each message is not constant, even though the network is not saturated. The discarded messages and nonconstant time delay may make Ethernet unsuitable in this loading situation for control applications.

Table 2 compares the performance of three releasing policies for a message period of 5000 μs. The network efficiency is low for the zero and random releasing policies, but can reach 100% in the scheduled releasing policy if the traffic load is not saturated. With a message period of 5000 μs, none of the networks is saturated, and the network utilizations of ControlNet and DeviceNet are the same for the three releasing policies. For Ethernet, however, the network utilization in the zero and random releasing policies is different from that in the scheduled releasing policy because there are multiple transmissions for one message when messages collide.

Again using the 5000 μs period, we also demonstrate the time delay history of the messages sent by three different nodes on the network (nodes 1, 5, and 10 in Fig. 9). DeviceNet is the only network that can guarantee constant time delays for all three releasing policies; this is due to the priority arbitration mechanism and the periodicity of

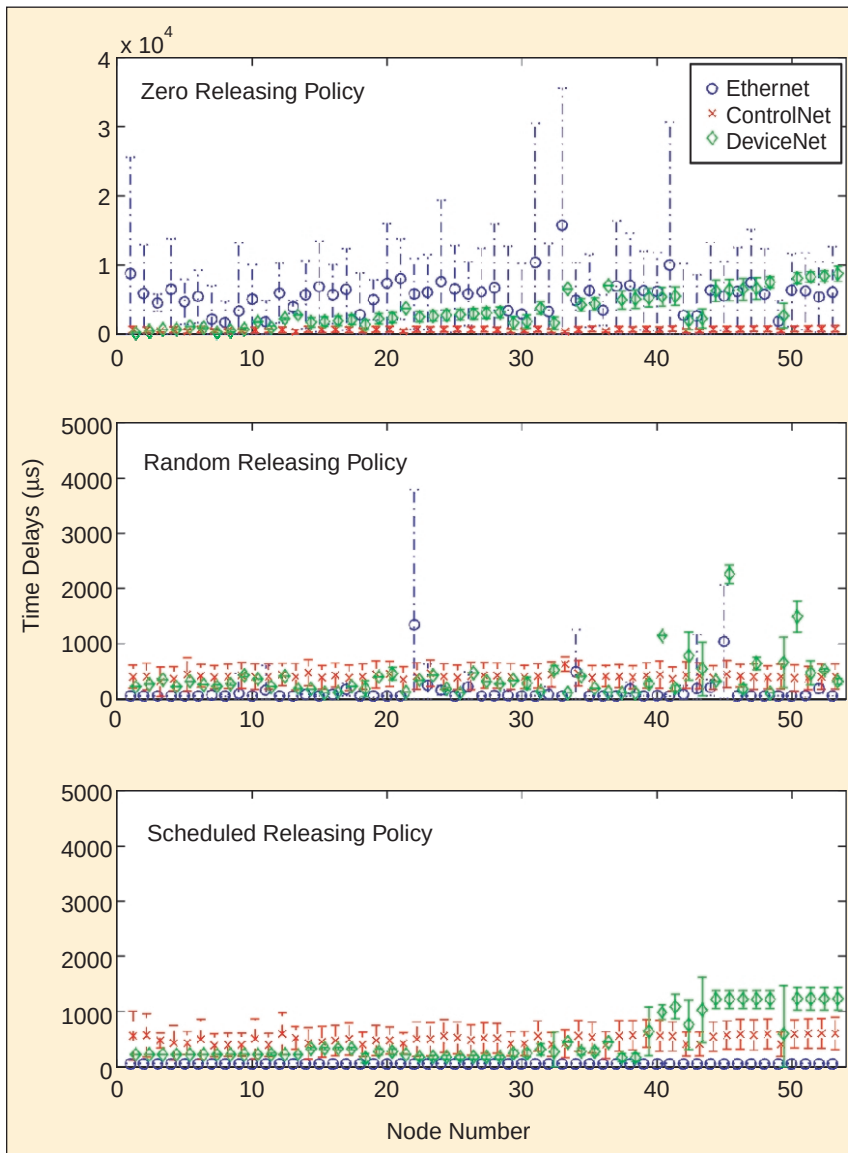


messages. Hence, the qualitative performance of DeviceNet is independent of the releasing policy in this case (i.e., when the network is not saturated). As the traffic load increases (high frequency of messages), only higher priority nodes can gain access to the network and maintain bounded transmission time delays, but low-priority messages cannot access the network and remain unsent. Using the scheduled and random releasing policies, the Ethernet message collision probability is low at low message rates. Hence, Ethernet generally has a constant time delay in low traffic loads. When collisions occur as in the zero or random releasing policy, however, the time delay is not constant and is difficult to predict, as shown in Fig. 9(a) and (b). Although ControlNet only exhibits a constant time delay under scheduled releasing policy conditions, it always guarantees a bounded time delay when the traffic load is not saturated, no matter what the releasing policy, as shown in Fig. 9.

The delays experienced by all the messages on the network are combined in Fig. 10. Again, with a message period of 5000  $\mu$ s, none of the networks is saturated. In Ethernet, shown in Fig. 10(a), the zero and random releasing policies demonstrate their nondeterministic time delays, even though the traffic load is not saturated. Fig. 10(b) shows that the message time delay of ControlNet is bounded for all releasing policies; we can estimate the lower and upper bounds based on the formulas derived in the timing analysis section. Due to the asynchronicity between the message period and the token rotation period, these time delays exhibit a linear trend with respect to the message number. The simulation results for DeviceNet, shown in Fig. 10(c), demonstrate that every node in DeviceNet has a constant time delay that depends only on the message number. The estimated mean time delay (1091  $\mu$ s) for Ethernet in Fig. 10(a) is computed for the case of the zero releasing policy from (2)

Table 3. SAE Benchmark application. The data size (bytes) and message period (ms) of the 53 nodes. Not all nodes have the same period, but each node has its own constant period for messages to be sent. These 53 nodes can be grouped into six groups by their message periods, which are 5 ms (eight nodes), 10 ms (two nodes), 20 ms (one node), 50 ms (30 nodes), 100 ms (six nodes), and 1000 ms (six nodes).

| Node | Data Sizes (bytes) | Period (ms) | Node | Data Size (bytes) | Period (ms) | Node | Data Size (bytes) | Period (ms) |
|------|--------------------|-------------|------|-------------------|-------------|------|-------------------|-------------|
| 1    | 8                  | 100         | 21   | 2                 | 1000        | 41   | 1                 | 50          |
| 2    | 8                  | 100         | 22   | 3                 | 50          | 42   | 8                 | 5           |
| 3    | 8                  | 1000        | 23   | 1                 | 50          | 43   | 8                 | 5           |
| 4    | 8                  | 100         | 24   | 1                 | 50          | 44   | 1                 | 50          |
| 5    | 8                  | 1000        | 25   | 1                 | 50          | 45   | 1                 | 50          |
| 6    | 8                  | 100         | 26   | 1                 | 50          | 46   | 1                 | 50          |
| 7    | 8                  | 5           | 27   | 1                 | 50          | 47   | 1                 | 50          |
| 8    | 8                  | 5           | 28   | 1                 | 50          | 48   | 1                 | 50          |
| 9    | 8                  | 5           | 29   | 8                 | 10          | 49   | 8                 | 5           |
| 10   | 8                  | 100         | 30   | 8                 | 10          | 50   | 2                 | 50          |
| 11   | 8                  | 5           | 31   | 2                 | 50          | 51   | 8                 | 50          |
| 12   | 8                  | 100         | 32   | 8                 | 5           | 52   | 1                 | 50          |
| 13   | 1                  | 1000        | 33   | 1                 | 1000        | 53   | 8                 | 50          |
| 14   | 4                  | 50          | 34   | 8                 | 50          |      |                   |             |
| 15   | 1                  | 50          | 35   | 1                 | 50          |      |                   |             |
| 16   | 1                  | 50          | 36   | 2                 | 1000        |      |                   |             |
| 17   | 2                  | 50          | 37   | 1                 | 50          |      |                   |             |
| 18   | 1                  | 20          | 38   | 1                 | 50          |      |                   |             |
| 19   | 1                  | 50          | 39   | 7                 | 50          |      |                   |             |
| 20   | 3                  | 50          | 40   | 1                 | 50          |      |                   |             |



**Figure 11.** Statistical demonstration of message time delay on three control networks for the SAE example: node numbering by subsystem. Note: The symbols “○,” “×” and “◇” denote the message time delay means of Ethernet, ControlNet, and DeviceNet, respectively; the dashdot, dashed, and solid lines represent the amount of message time delay variance computed from the simulation result as twice the standard deviation of the time delay.

and a two standard deviation bound is also shown. This estimated value is close to the simulated value (1081  $\mu$ s) given in Table 2. The maximum and minimum time delays for ControlNet and DeviceNet are computed from (3), (4), and (6).

## Case Study II: SAE Benchmark

Next we consider a benchmark application from the SAE [25]. We simulate the specified traffic flow on three control networks. In this example, there are 53 nodes that belong to seven subsystems, namely: the batteries, the vehicle controller, the inverter/motor controller, the instrument panel dis-

play, the driver inputs, the brakes, and the transmission control. Not all nodes have the same period, but each node has its own constant period of messages to be sent. Because the minimum data element in all three networks is 1 byte, a message of 1 or 8 bits has the same transmission time. To study the impact of data size on network performance, we increase the data size at each node by a factor of eight (e.g., changing node 1 from 8 bits in [25] to 8 bytes.) The data size and period of each node are shown in Table 3. We also apply the three releasing policies (zero, random, and scheduled) as discussed earlier. The total simulation time is 10 s. The simulation results are shown in Figs. 11 and 12, and a summary is given in Table 4.

Table 4 illustrates three cases based on three different node numbering assignments: the node numbering in Case I is ordered based on the functional groups, and the other two cases are ordered by message periods (Case II is decreasing, and Case III is increasing). Note that the total transmission times for all 53 messages are 3052.8, 1366.4, and 8470  $\mu$ s in Ethernet, ControlNet, and DeviceNet, respectively.

Case I simulation results for the three control networks are compared in Fig. 11. For the zero releasing policy, Ethernet has the largest message time delay value and standard deviation over all 53 nodes. This is due to the high probability of message collision with the zero releasing policy. For DeviceNet, as expected, there is a linear trend for message time delay over the message node numbers. For ControlNet, however, the average message time delay is nearly the same over all 53 nodes. This is because the MAC in ControlNet is based on the token rotation mechanism, and every node gains access to the network medium at a deterministic period. Fig. 11 also shows lower and consistent variability of time delays for ControlNet and DeviceNet compared to that for Ethernet.

For the random releasing policy, ControlNet still demonstrates a consistent time delay over all 53 nodes, but Ethernet and DeviceNet do not. For DeviceNet, however, nodes with small node numbers still have a shorter message time delay than nodes with large node numbers. The average message time delays of the random releasing policy are

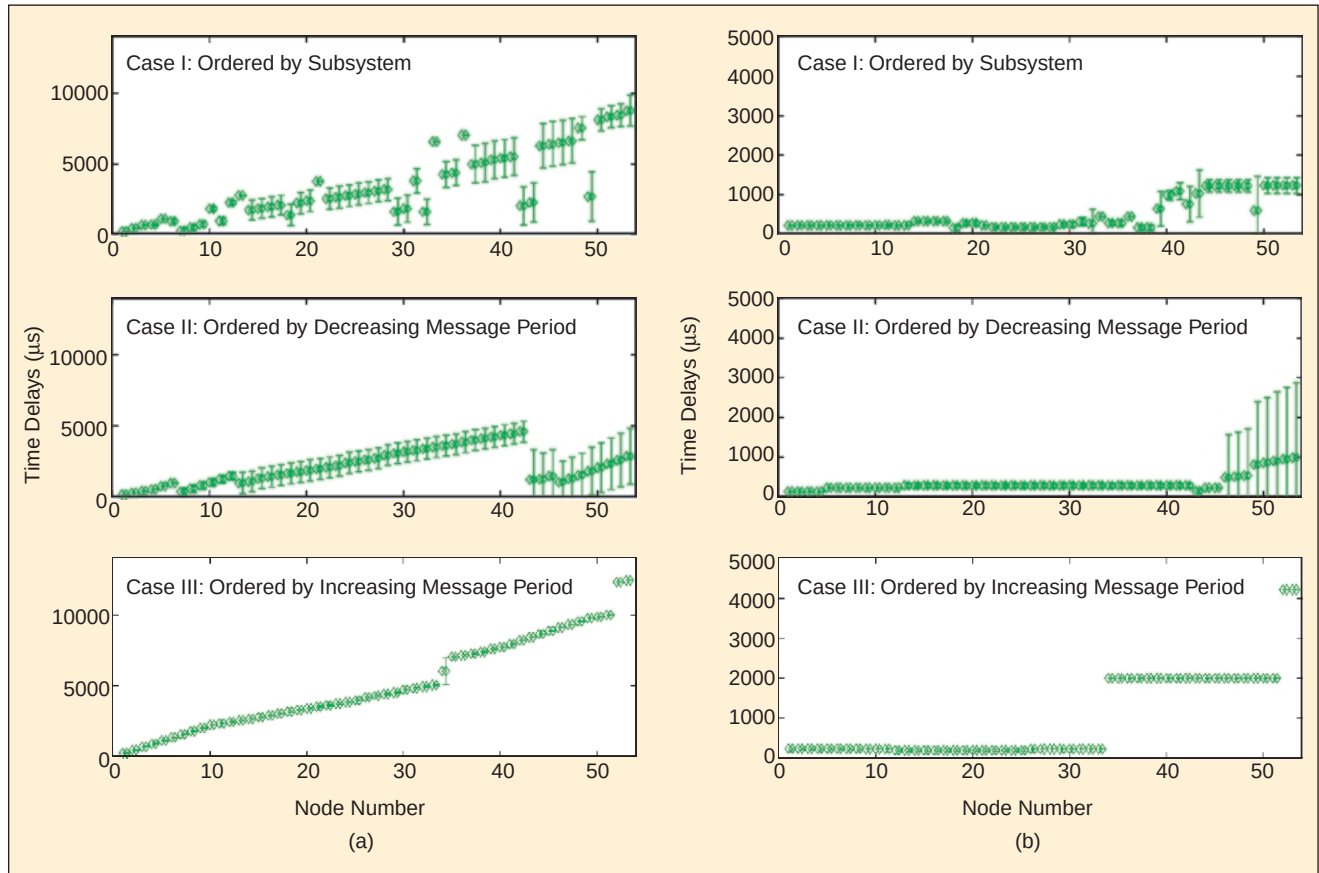
shorter than those of the zero releasing policy, as shown in Table 4. Ethernet still has high variability of time delay compared to ControlNet and DeviceNet.

For the scheduled releasing policy, Ethernet demonstrates outstanding results because of its high transmission speed and zero possibility of message collisions. Because the total transmission time of all 53 messages is  $8470\text{ }\mu\text{s}$  in DeviceNet, the last few nodes with short message periods (e.g., nodes 39-53) have higher variability of message time delays, especially nodes 39, 42, 43, and 49, which have large data sizes and shorter message periods. The results for ControlNet, however, are similar for all releasing policies. For multiperiod systems, ControlNet is less sensitive to the releasing policies, and it is very difficult to coordinate the system parameters and maintain the minimum time delay, as we did in the ten-node simulation.

In this SAE example setting, shown in Table 3, there are six different message periods. To study the effect of node numbering, we regroup these nodes by their message periods. We first let those nodes with shorter message periods have lower node numbers and find that the nodes within one group have similar performance for the ControlNet and DeviceNet systems but that performance varies between node groups. There is not much difference in per-

formance between node groups for Ethernet, however. Note that the simulation results within one group are similar to the ten-node example we studied in the first part of this section.

Fig. 12 shows the time delay characteristics of two releasing policies in DeviceNet for three cases of node numbering. Because of the priority mechanism on node numbers, the time delay profile exhibits a linear trend over node numbers. There is little variance, however, for the case in which messages are ordered by increasing period in the zero releasing policy. This is because high-frequency nodes are high-priority nodes and have little difficulty accessing the network medium. For the scheduled releasing policy, both the average time delay and the variance are smaller than for the zero releasing policy. The last few nodes of the decreasing message period numbering case have large time delay variance with both releasing policies because they are lower priority nodes with high-frequency messages and always lose the contention battle when one arises. The gap between nodes 33 and 34 is due to the fact that the total transmission time from nodes 1 to 33 is more than  $5000\text{ }\mu\text{s}$  and node 34 always loses the contention to those nodes with higher priority. A similar situation occurs between nodes 51 and 52.



**Figure 12.** Statistical demonstration of message time delay on DeviceNet for the SAE example. Note: The symbol “ ” denotes the message time delay mean; the solid lines represent the amount of message time delay variance computed from the simulation result as twice the standard deviation of the time delay; (a) zero-releasing policy, (b) scheduled releasing policy.

| Table 4. Simulation results of SAE example: Three node numbering cases. |                              |       |       |                                                  |       |       |                                                   |       |       |
|-------------------------------------------------------------------------|------------------------------|-------|-------|--------------------------------------------------|-------|-------|---------------------------------------------------|-------|-------|
| Node Numbering                                                          | Case I<br>Subsystem Ordering |       |       | Case II<br>Decreasing Message Period<br>Ordering |       |       | Case III<br>Increasing Message Period<br>Ordering |       |       |
| Releasing Policy                                                        | Zero                         | Rand. | Sch'd | Zero                                             | Rand. | Sch'd | Zero                                              | Rand. | Sch'd |
| <b>Average time delay (<math>\mu</math>s)</b>                           |                              |       |       |                                                  |       |       |                                                   |       |       |
| Ethernet                                                                | 3268                         | 122   | 58    | 3377                                             | 103   | 58    | 3362                                              | 80    | 58    |
| ControlNet                                                              | 558                          | 417   | 439   | 555                                              | 406   | 458   | 562                                               | 407   | 444   |
| DeviceNet                                                               | 2176                         | 428   | 453   | 2021                                             | 305   | 568   | 2202                                              | 253   | 386   |
| <b>Efficiency (%)</b>                                                   |                              |       |       |                                                  |       |       |                                                   |       |       |
| Ethernet                                                                | 1.7                          | 47.2  | 100.0 | 1.8                                              | 55.8  | 100.0 | 1.8                                               | 71.8  | 100.0 |
| ControlNet                                                              | 5.6                          | 7.5   | 7.1   | 5.6                                              | 7.7   | 6.8   | 5.6                                               | 7.7   | 7.0   |
| DeviceNet                                                               | 9.1                          | 46.2  | 43.7  | 9.8                                              | 64.8  | 34.8  | 9.0                                               | 78.1  | 51.2  |
| <b>Utilization (%)</b>                                                  |                              |       |       |                                                  |       |       |                                                   |       |       |
| Ethernet                                                                | 47.6                         | 16.6  | 14.5  | 47.5                                             | 16.3  | 14.5  | 47.5                                              | 15.4  | 14.5  |
| ControlNet                                                              | 7.9                          | 7.9   | 7.9   | 7.9                                              | 7.9   | 7.9   | 7.9                                               | 7.9   | 7.9   |
| DeviceNet                                                               | 49.7                         | 49.7  | 49.7  | 49.7                                             | 49.7  | 49.7  | 49.7                                              | 49.7  | 49.7  |

## Conclusions and Future Work

The features of three candidate control networks-Ethernet (CSMA/CD), ControlNet (token bus), and DeviceNet (CAN)-were discussed in detail. We first described the MAC mechanisms of these protocols, which are responsible for satisfying both the time-critical/real-time response requirement over the network and the quality and reliability of communication between devices on the network. These timing parameters, which will ultimately influence control applications, are affected by the network data rate, the periods of messages, the data or message size of the information, and the communication protocol. For each protocol, we studied the key performance parameters of the corresponding network when used in a control situation, including network utilization, magnitude of the expected time delay, and characteristics of time delays. Simulation results were presented for several different scenarios. The timing analyses and comparisons of message time delay given in this article should be useful for designers of NCSs.

The evaluation study described here is based on the characteristics and requirements of control systems. The characteristics are small data sizes and periodic messages, and the requirements include guaranteed transmission and bounded time delay. The suitability of network protocols for use in control systems is greatly influenced by these two criteria. Although Ethernet has seen widespread use in many data transmission applications and can support high data rates up to 1

Gb/s, it may not be suitable as the communication medium for some control systems when compared with deterministic network systems. Because of its high data rate, however, Ethernet can be used for aperiodic/non-time-critical and large data size communication, such as communication between workstations or machine cells. For machine-level communication with controllers, sensors, and actuators, deterministic networks are generally more suitable for meeting the characteristics and requirements of control systems. For control systems with short and/or prioritized messages, DeviceNet demonstrates better performance. The scheduled and unscheduled messaging capabilities in ControlNet make it suitable for time-critical and non-time-critical messages. ControlNet is also suitable for large data size message transmission.

Our future efforts will focus on controller design for NCSs, which can differ significantly from the design of traditional centralized control systems. This work will include conducting experimental studies of control networks for control applications. We plan to use this analysis, along with the performance evaluation of candidate control networks presented here, as the basis for future message scheduling and control algorithm designs for NCSs.

## Acknowledgments

We wish to acknowledge the contributions of J. Korsakas, the support of the National Science Foundation Engineering Re-

search Center for Reconfigurable Machining Systems at the University of Michigan under grant EEC95-92125, the Open DeviceNet Vendor Association, and ControlNet International.

## References

- [1] L.H. Eccles, "A smart sensor bus for data acquisition," *Sensors*, vol. 15, no. 3, pp. 28-36, 1998.
- [2] S. Biegacki and D. VanGompel, "The application of DeviceNet in process control," *ISA Trans.*, vol. 35, no. 2, pp. 169-176, 1996.
- [3] L.D. Gibson, "Autonomous control with peer-to-peer I/O networks," *Sensors*, vol. 12, no. 9, pp. 83-90, Sept. 1995.
- [4] A. Ray, "Introduction to networking for integrated control systems," *IEEE Contr. Syst. Mag.*, vol. 9, pp. 76-79, Jan. 1989.
- [5] G. Schickhuber and O. McCarthy, "Distributed fieldbus and control network systems," *Computing Contr. Eng.*, vol. 8, no. 1, pp. 21-32, Feb. 1997.
- [6] D. Song, T. Divoux, and F. Lepage, "Design of the distributed architecture of a machine-tool using FIP fieldbus," in *Proc. IEEE Int. Conf. Application Specific Systems, Architectures, Processors*, Los Alamitos, CA, 1996, pp. 250-260.
- [7] R.S. Raji, "Smart networks for control," *IEEE Spectrum*, vol. 31, pp. 49-55, June 1994.
- [8] Y. Koren, Z.J. Pasek, A.G. Ulsoy, and U. Benchetrit, "Real-time open control architectures and system performance," *CIRP Annals-Manuf. Technol.*, vol. 45, no. 1, pp. 377-380, 1996.
- [9] *DeviceNet Specifications*, 2nd. ed. Boca Raton, FL: Open DeviceNet Vendors Association, 1997.
- [10] G. Cena, C. Demartini, and A. Valenzano, "On the performances of two popular fieldbuses," in *Proc. IEEE Int. Workshop Factory Commun. Syst.*, Barcelona, Spain, Oct. 1997, pp. 177-186.
- [11] J.D. Wheelis, "Process control communications: Token Bus, CSMA/CD, or Token Ring?" *ISA Trans.*, vol. 32, no. 2, pp. 193-198, July 1993.
- [12] *ControlNet Specifications*, 2nd ed. Boca Raton, FL: ControlNet International, 1998.
- [13] S. Saad-Bouzeffrane and F. Cottet, "A performance analysis of distributed hard real-time applications," in *Proc. IEEE Int. Workshop Factory Commun. Syst.*, Barcelona, Spain, Oct. 1997, pp. 167-176.
- [14] S.A. Koubias and G.D. Papadopoulos, "Modern fieldbus communication architectures for real-time industrial applications," *Comput. Ind.*, vol. 26, no. 3, pp. 243-252, Aug. 1995.
- [15] D. Bertsekas and R. Gallager, *Data Networks*, 2nd ed. Englewood Cliffs, NJ: Prentice-Hall, 1992.
- [16] B.J. Casey, "Implementing Ethernet in the industrial environment," in *Proc. IEEE Ind. Applicat. Soc. Ann. Meeting*, vol. 2, Seattle, WA, Oct. 1990, pp. 1469-1477.
- [17] A.S. Tanenbaum, *Computer Networks*, 3rd ed. Upper Saddle River, NJ: Prentice-Hall, 1996.
- [18] H. Bartlett and J. Harvey, "The modeling and simulation of a pick and place computer-integrated manufacturing (CIM) cell," *Comput. Ind.*, vol. 26, no. 3, pp. 253-260, Aug. 1995.
- [19] G. Paula, "Building a better fieldbus," *Mech. Eng.*, pp. 90-92, June 1997.
- [20] V.K. Khanna and S. Singh, "An improved 'piggyback Ethernet' protocol and its analysis," *Comput. Networks ISDN Syst.*, vol. 26, no. 11, pp. 1437-1446, Aug. 1994.
- [21] K.K. Ramakrishnan and H. Yang, "The Ethernet capture effect: Analysis and solution," in *Proc. 19th Conf. Local Comp. Networks*, Minneapolis, MN, Oct. 1994, pp. 228-240.
- [22] J. Eidson and W. Cole, "Ethernet rules closed-loop system," *InTech*, pp. 39-42, June 1998.
- [23] J. Moyne, N. Najafi, D. Judd, and A. Stock, "Analysis of sensor/actuator bus interoperability standard alternatives for semiconductor manufacturing," in *Sensors Expo Conf. Proc.*, Cleveland, OH, Sept. 1994.
- [24] F.-L. Lian, J. Moyne, and D.M. Tilbury, "Performance evaluation of control networks: Ethernet, ControlNet, and DeviceNet," Tech. Rep. UM-MEAM-99-02, Feb. 1999. Available: <http://www.eecs.umich.edu/~impact>
- [25] K. Tindell, A. Burns, and A.J. Wellings, "Calculating controller area network (CAN) message response times," *Contr. Eng. Practice*, vol. 3, no. 8, pp. 1163-1169, Aug. 1995.
- [26] F.-L. Lian, J. Moyne, and D.M. Tilbury, "Control performance study of a networked machining cell," in *Proc. American Control Conf.*, Chicago, IL, June 2000, pp. 2337-2341.

**Feng-Li Lian** received the B.S. degree in agricultural machinery engineering in 1992 and the M.S. degree in electrical engineering in 1994 from National Taiwan University and the M.S. degree in industrial and operations engineering in 1998 from the University of Michigan. Currently he is a Ph.D. student in the Department of Mechanical Engineering at the University of Michigan. His areas of research include networked control systems, constrained flexible manipulators, and statistical process control.

**James Moyne** received his B.S.E.E. and B.S.E. degrees in math and his M.S.E.E. and Ph.D. degrees from the University of Michigan. Since 1992 he has been an Associate Research Scientist in the Department of Electrical Engineering and Computer Science at the University of Michigan. He is also President and co-founder of MiTeX Solutions, Inc. His areas of research include advanced process control, database technology, and sensor bus technology. He has written a number of refereed publications in each of these areas. He also holds a patent and is the author of several semiconductor manufacturing standards.

**Dawn M. Tilbury** received the B.S. degree in electrical engineering *summa cum laude* from the University of Minnesota in 1989 and the M.S. and Ph.D. degrees in electrical engineering and computer sciences from the University of California, Berkeley, in 1992 and 1994, respectively. Since 1995 she has been an Assistant Professor in the Mechanical Engineering Department at the University of Michigan, Ann Arbor. She received an Undergraduate Computational Engineering and Science Award from the U.S. Department of Energy in 1995 and the EDUCOM Medal (jointly with Prof. Messner) in 1997. Her research interests include trajectory planning for nonlinear systems (including mobile robots), real-time distributed control of mechanical systems, and logic control of manufacturing systems. She received an NSF CAREER award in 1999. She is a member of the IEEE, ASME, and SWE.